Fiber Nonlinear Compensation Algorithm

Python Code Documentation (Stage 3)

Agreement No.: YBN2018105241

April 27, 2020

Novosibirsk State University

1 Libraries

Import all necessary libraries and packages.

```
from mxnet import nd, gluon, init, autograd, gpu
from mxnet.gluon import nn
import mxnet as mx

import h5py
import numpy as np
import math
import time
import datetime
from os import makedirs
import os
```

Library `mxnet` and all included packages contains all the necessary tools for design and train neural networks. `h5py` is used to load data from `.mat` files. Libraries `numpy` and `math` are for working with arrays and mathematical operations. `time` and `datetime` are for manipulating dates and times. Module `os` is used to to create folders and save files.

2 Constants

Define paths for data and result files. Set Gray encoding and precision for floating-point arithmetic.

```
root = ''
file_name = 'Data_for_NSU_32v2.mat'

results_path = ''
```

Variable `root` contains the path to the Matlab data file. If it is empty, then the data file should be in the same folder as the Jupyter Notebook file. Variable `file_name` corresponds to the Matlab data file name. Variable `results_path` contains the path to the folder where model and parameters of calculations will be saved.

```
gray_symbols_16qam = np.sqrt(0.1) * np.array(
                     [1+1j, 1+3j, 1-1j, 1-3j,
                     3+1j, 3+3j, 3-1j, 3-3j,
                     -1+1j, -1+3j, -1-1j, -1-3j,
                     -3+1j, -3+3j, -3-1j, -3-3j])
```

This code block corresponds to to the array with the Gray encoding used. Symbols are normalized so that the average power is equal to one.

```
swap_64_to_128_complex = False

if swap_64_to_128_complex:
    complex_t = np.complex128
    real_t = np.float64
else:
    complex_t = np.complex64
    real_t = np.float32
```

Variable `swap_64_to_128_complex` set precision for floating-point arithmetic. If it is equal to `False`, then single-precision floating-point format is used (32-bit float). When `swap_64_to_128_complex = True`, all calculations are performed in double precision (64-bit float).

3 Load .mat file

Load data and parameters from the Matlab file provided by Huawei.

```
with h5py.File(root + file_name, 'r') as f:
    BER_CDC_X = np.array(f['BER_CDC_X'])[0]
    BER_CDC_Y = np.array(f['BER_CDC_Y'])[0]
    BER_DBP_X = np.array(f['BER_DBP_X'])[0]
    BER_DBP_Y = np.array(f['BER_DBP_Y'])[0]

    RX_CDC = np.asarray((f['RX_CDC'][()])['real'], dtype=real_t) +
        1j * np.asarray((f['RX_CDC'][()])['imag'], dtype=real_t)
    RY_CDC = np.asarray((f['RY_CDC'][()])['real'], dtype=real_t) +
        1j * np.asarray((f['RY_CDC'][()])['imag'], dtype=real_t)
    TX = np.asarray((f['TX'][()])['real'], dtype=real_t) +
        1j * np.asarray((f['TX'][()])['imag'], dtype=real_t)
    TY = np.asarray((f['TY'][()])['real'], dtype=real_t) +
        1j * np.asarray((f['TY'][()])['imag'], dtype=real_t)

    Dcd = (f['param_ch']['Dcd'])[()].item()
    fiberL = (f['param_ch']['fiberL'])[()].item()
    gamma = (f['param_ch']['gamma'])[()].item()
    lmbda = (f['param_ch']['lambda'])[()].item()

    Fn = np.array(f['param_gen']['Fn']).T[0]
    Fsym = (f['param_gen']['Fsym'])[()].item()
    Po = (f['param_gen']['Po'])[()].item()
```

This code block corresponds to loading all the necessary arrays and scalars from a file with a name `file_name` and located in `root` using h5py package. The names of the created variables coincide with the names from the Matlab file.

```
Dcd = Dcd * 1e-6
fiberL = fiberL * 1e3
lmbda = lmbda * 1e-9
gamma = gamma * 1e-3
```

In this block, the variables are transferred to the SI system.

```
c = 299792458
b2 = -lmbda ** 2 * Dcd / ( 2 * math.pi * c)
power = 10 ** ((Po-30)/10) / 2
```

In these lines of code, the variable `b2` (β_2) is calculated using the dispersion parameter `Dcd` and the power is converted from dB (variable `Po`) to W (variable `power`).

4 Variables

Set user-defined and calculated variables.

4.1 Defined

Set variables defined by user.

```
ctx = gpu(0)
```

This line is responsible for choosing the device on which the calculations will be performed. By default, the code will use GPU. To run calculations on the CPU, this line must be replaced by `ctx = mx.cpu(0)`.

```
channel_numbers = [0, 1, 2, 3]

train_portion = 0.5
CDC_data_size = 2 ** 17
```

Array `channel_numbers` contains the channel numbers to be processed. Numbering starts at 0. Variable `train_portion` sets which part of the data (arrays `RX_CDC, RY_CDC, TX, TY`)

will be used to train the neural network. Variable `CDC_data_size` determines the size of the training set for preliminary calculation of CDC filter coefficients.

```
num_step = 19
filt_CDC_width = 101
```

Variable `num_step` determines the number of NN layers, each of which includes a convolution layer for CD compensation and an activation function layer corresponding to compensation for nonlinear effects. Variable `filt_CDC_width` sets the width of the convolution filter (the number of coefficients) for CD compensation at each step.

```
manual_shift = True
filt_CDC_last_width = 71
filt_FD_width = 21
```

Variable `manual_shift` defines the procedure for convolution filters training. In the case when the signal propagates at a frequency different from the carrier frequency, two approaches can be used to find the CDC filter coefficients. In the first case (`manual_shift = False`), the filter coefficients can be found in a straightforward manner, similar to the procedure that applies if the signal propagates at the carrier frequency. However, in this case the filter impulse response is shifted and we have to use a larger number of weights some of which are close to zero. In the second approach (`manual_shift = True`), we first compensate CD as if the signal propagates at the carrier frequency and next perform a time shift by the corresponding value. As a result we compensate for the chromatic dispersion as if the signal propagated at the frequency different from the carrier frequency. However, as we work with discrete-time signals, we need to tune the step length so that the time shift was divisible by the duration of the symbol interval. Since after `num_step` steps of this length the remaining fiber length can still correspond to a time shift greater than symbol interval, we need to use additional convolution layer to compensate for the dispersion along the length corresponding to the remaining integer number of time shifts. The width of such a filter (we call it last CDC filter) is set by the parameter `filt_CDC_last_width`. However, the time shift corresponding to the full length of the fiber is not divisible by the duration of the symbol interval at given channel frequencies (± 37.5 and ± 112.5 GHz). Therefore, a fractional delay filter (FD) is additionally used after the last step to account for any remaining non-integer delay. Variable `filt_FD_width` determines the number of FD filter coefficients.

```
nl_mem = 6
nl_mem_nc1_1 = 3
nl_mem_nc1_2 = 17
nl_mem_nc2_1 = 1
nl_mem_nc2_2 = 24
nl_mem_nc3_1 = 1
nl_mem_nc3_2 = 11
```

This code block defines the number of coefficients for convolution filters of the enhanced SSFM activation function. Here we generally use 4 groups of "nonlinear" filters: one to compensate for SPM effects, the other three for XPM compensation – for the closest adjacent spectral channels (e.g. channel 0 and channel 1), for spectral channels spaced at one channel spacing (e.g. channel 0 and channel 2) and spaced at two channel spacing (channel 0 and channel 3). We separate nonlinear filters because it makes sense to use a different number of symbols for channels located at different spectral distances. In addition, since the signals in different channels move in time relative to each other, it is necessary to use a different number of symbols to the left and to the right of the considered one in order to effectively take into account nonlinear interactions. Variable `nl_mem` sets the number of symbols in each direction (`nl_mem` left and `nl_mem` right neighboring symbols) used for a nonlinear filter to compensate for SPM effects. Variable `nl_mem_nc1_1` defines the number of symbols located in the direction of the time shift of the neighboring channel relative to the considered one, which are used for a nonlinear filter for the closest adjacent spectral channels. Variable `nl_mem_nc1_2` sets the number of symbols used in the opposite direction to the time shift of the adjacent spectral channel for the same filter. Variables `nl_mem_nc2_1`, `nl_mem_nc2_2`, `nl_mem_nc3_1`, `nl_mem_nc3_2` are similar to the previous two, except that they are used for the other two XPM filter groups.

```
epochs = 2000
batch_size = 200000
lrate = 1e-3
lrate_CDC = 1e-3
```

Variable `epochs` determines how many epochs should be performed when training a neural network. Variable `batch_size` sets the size of one batch. Variables define learning rates for training the total neural network and convolution filters for CD compensation, respectively.

```
sym_filt = True
sym_nonlin = True
```

Parameters `sym_filt` and `sym_nonlin` determine whether symmetric filters for CDC and nonlinear filters for SPM will be used (`True`) or not (`False`).

```
save_model_best = True
model_filename = "net.params"
```

Variable `save_model_best` defines whether the model with the minimum losses will be saved during training and used later for testing (`save_model_best = True`), or whether the model obtained after the last performed epoch will be used to calculate BER over the entire data set (`save_model_best = False`). Variable `model_filename` sets the name of the file with the saved model.

4.2 Calculated

Calculate auxiliary parameters.

```
channels = len(channel_numbers)
data_dimension = 2 * 2 * channels
```

Variable `channels` defines the number of channels to be processed. Variable `data_dimension` determines the dimension of the data array. The first factor 2 corresponds to the real and imaginary parts, the second factor 2 corresponds to two polarizations.

```
central_frequency = np.sum(Fn[channel_numbers]) / channels
Fn = Fn - central_frequency
```

This code block corresponds to the calculation of the new central frequency for the spectral channels under consideration (variable `central_frequency`) and the change of frequencies of all channels taking into account the new central frequency.

```
data_size = RX_CDC.shape[1]
train_data_size = int(data_size * train_portion)
CDC_data_size = min(CDC_data_size, train_data_size)
```

Variable `data_size` sets the size of the entire data set. Also here is the calculation of the size of the training set (variable `train_data_size`) and updating the variable `CDC_data_size` if `train_data_size` $<$ `CDC_data_size`.

```
filt_CDC_delay = int((filt_CDC_width-1)/2)
filt_CDC_last_delay = int((filt_CDC_last_width-1)/2)
filt_FD_delay = int((filt_FD_width-1)/2)
```

After applying a convolution with S coefficients, the data size will decrease by $S-1$ symbols. Here, the number of discarded symbols on each side of the data array after applying the CDC filter at each step, the last CDC filter and the FD filter is calculated. We call this parameter delay.

```
if sym_filt:
    filt_CDC_width = int((filt_CDC_width + 1) / 2)
    filt_CDC_last_width = int((filt_CDC_last_width + 1) / 2)
```

If `sym_filt = True` symbols to the left and to the right of the central one have the same weight and therefore the number of unknown coefficients will decrease accordingly..

```
symbol_shift = (fiberL / num_step * 2 * math.pi * b2 * Fsym *
    Fn[channel_numbers]).astype(int)
```

For a signal shifted by the frequency f_c, a time shift by single symbol interval will correspond to propagation by a distance of $1/(2\pi\beta_2 R_s f_c)$, where R_s is the symbol rate. Array `symbol_shift` corresponds to the integer number of symbols by which each of the considered channels will be shifted when propagating by one step (`fiberL/num_step`).

```
if np.max(symbol_shift) == 0 or not manual_shift:
    step_length = fiberL / num_step
    last_step_length = 0
    FD_step_length = 0
```

If only one channel is propagated or `manual_shift = False`, then the step length for the CDC filter (variable `step_length`) is `fiberL/num_step` and the step lengths for the last CDC and FD filters will be zero.

```
else:
    symbol_shift_distance = np.max(abs(1/(2 * math.pi * b2 *
        Fsym * Fn[channel_numbers])))
    ind = np.argmax(abs(1/(2 * math.pi * b2 * Fsym * Fn[channel_numbers])))

    step_length = symbol_shift[ind] * symbol_shift_distance
    symbol_shift = (step_length * 2 * math.pi * b2 * Fsym *
        Fn[channel_numbers]).astype(int)

```

```
    last_symbol_shift = ((fiberL - step_length * num_step) * 2 * math.pi * b2 *
        Fsym * Fn[channel_numbers]).astype(int)
    last_step_length = last_symbol_shift[ind] * symbol_shift_distance
    last_symbol_shift = (last_step_length * 2 * math.pi * b2 * Fsym *
        Fn[channel_numbers]).astype(int)

    FD_step_length = fiberL - step_length * num_step - last_step_length
```

Otherwise, first the maximum distance among all channels under consideration at which a shift by one symbol occurs is calculated (variable `symbol_shift_distance`). Variable `ind` corresponds to the number of such a channel. Next, in line 6, the length of one step is calculated. Thus, `step_length` is the maximum possible step length such that the corresponding time shift for each channel is divisible by the symbol duration and `step_length` $\leq$ `fiberL/num_step`. Line 7 redefines the array `symbol_shift` to reflect the new step length. Variables `last_symbol_shift` and `last_step_length` is similar to `symbol_shift` and `step_length` except that instead of the length of one step, the remainder of the fiber length after `num_step` steps is considered. Variable `FD_step_length` defines the residual fiber length for the FD filter.

```
step_shifts = np.zeros((channels,2),dtype=int)
if last_step_length > 0:
    last_shifts = np.zeros((channels,2),dtype=int)
```

If `manual_shift = True`, in order to shift the signals in different frequency channels, we discard some of the first symbols and some of the last symbols of arrays obtained after CDC filters. Array `step_shifts` contains the number of first and last symbols that are discarded after applying CDC filters at each step for all processed channels. Array `last_shifts` is the same as `step_shifts`, but for the last CDC filter. In this block of code, these arrays are initialized.

```
for i in range(channels):
    if symbol_shift[i] > 0:
        step_shifts[i,0] = abs(np.min(symbol_shift)) - symbol_shift[i]
        step_shifts[i,1] = np.max(symbol_shift) + symbol_shift[i]
    else:
        step_shifts[i,0] = np.max(symbol_shift) - symbol_shift[i]
        step_shifts[i,1] = abs(np.min(symbol_shift)) + symbol_shift[i]

    if last_step_length > 0:
        if last_symbol_shift[i] > 0:
            last_shifts[i,0] = abs(np.min(last_symbol_shift)) - last_symbol_shift[i]
            last_shifts[i,1] = np.max(last_symbol_shift) + last_symbol_shift[i]
```

```
        else:
            last_shifts[i,0] = np.max(last_symbol_shift) - last_symbol_shift[i]
            last_shifts[i,1] = abs(np.min(last_symbol_shift)) + last_symbol_shift[i]
```

With the parameters used, in the case of 2 channels, array `symbol_shift` has the form [-a, a], and in the case of 4 channels – [-3a, -a, a, 3a] (for example, when `num_step` = 19, a = 27). When discarding symbols, it is necessary that the corresponding symbols on different spectral channels are opposite each other. For example, after one propagation step in the case of two channels, the symbol x_n from the first channel should be opposite the symbol y_{n-2a} from the second channel. In addition, the resulting arrays must be of the same length for all channels. Thus, in the case of 2 channels, for the first channel, we must discard the first 2a symbols, and for the second, the last 2a symbols. Then we comply both of these conditions. In the case of 4 channels reasoning in the same way, for the first channel we must discard the first 6a symbols, for the second channel – the first 4a symbols and 2a symbols at the end, for the third channel — the first 2a symbols and the last 4a symbols and for channel 4 we must discard the last 6a symbols. In this block, arrays `step_shifts` and `last_shifts` are filled in accordance with the considerations made.

```
step_output_shifts = np.zeros((2,),dtype=int)
if last_step_length > 0:
    last_output_shifts = np.zeros((2,),dtype=int)

step_output_shifts[0] = abs(np.min(symbol_shift))
step_output_shifts[1] = np.max(symbol_shift)
if last_step_length > 0:
    last_output_shifts[0] = abs(np.min(last_symbol_shift))
    last_output_shifts[1] = np.max(last_symbol_shift)
```

Since after the shift of symbols, the resulting array decreases, the size of the array containing the desired NN output should accordingly be reduced. This array should decrease symmetrically from both sides and by the same value as the arrays in which the shifts are performed. Arrays `step_output_shifts` and `last_output_shifts` contain the number of first and last symbols that should be discarded in desired NN output data for one CDC filter and the last CDC filter (for example, for single CDC filter in the case of 2 channels it will be [a,a] and in the case of 4 channels – [3a, 3a]). Since the data in all channels is reduced by the same number, these arrays can be used for any processed channel.

```
full_output_shifts = np.zeros((2,),dtype=int)
full_output_shifts[0] = num_step * abs(np.min(symbol_shift))
full_output_shifts[1] = num_step * np.max(symbol_shift)
if last_step_length > 0:
    full_output_shifts[0] += abs(np.min(last_symbol_shift))
    full_output_shifts[1] += np.max(last_symbol_shift)

max_full_shift = int(np.sum(full_output_shifts))
```

Array `full_output_shifts` by analogy with `step_output_shifts` and `last_output_shifts` contains the number of first and last symbols that should be discarded in desired output data of the entire neural network after `num_step` CDC filters and the last CDC filter. Variable `max_full_shift` determines the value by which the size of the array containing the desired NN output should be reduced.

```
nonlin_coef = gamma * power * 0.05 * 8/9 * step_length / channels
```

Variable `nonlin_coef` sets the initial approximation of the nonlinear parameter γ used on activation function layers. Its value was obtained using numerical experiments.

```
if sym_nonlin:
    memory_size = nl_mem + 1
else:
    memory_size = 2 * nl_mem + 1
memory_size_nc1 = nl_mem_nc1_1 + nl_mem_nc1_2 + 1
memory_size_nc2 = nl_mem_nc2_1 + nl_mem_nc2_2 + 1
memory_size_nc3 = nl_mem_nc3_1 + nl_mem_nc3_2 + 1
```

Variable `memory_size` determines the width (number of coefficients) of the nonlinear filter for SPM. If this filter is symmetric (`sym_nonlin = True`), then the number of unknown coefficients will be less, since the left and right neighboring symbols have the same coefficients. The remaining variables in this block define the width of the XPM filters for the closest adjacent spectral channels (XPM-1), for spectral channels spaced at one channel spacing (XPM-2) and spaced at two channel spacing (XPM-3).

```
nl_mem_nc = np.zeros((4,2),dtype=int)
nl_mem_nc[0,0] = nl_mem
nl_mem_nc[0,1] = nl_mem
nl_mem_nc[1,0] = nl_mem_nc1_1
nl_mem_nc[1,1] = nl_mem_nc1_2
nl_mem_nc[2,0] = nl_mem_nc2_1
nl_mem_nc[2,1] = nl_mem_nc2_2
nl_mem_nc[3,0] = nl_mem_nc3_1
```

```
nl_mem_nc[3,1] = nl_mem_nc3_2

max_nl_shift = np.max(nl_mem_nc[:channels,:])
```

In this block of code, helper variable `nl_mem_nc` containing the number of used left and right neighboring symbols for all nonlinear filters is declared. Variable `max_nl_shift` defines the maximum number of symbols used in one of the directions for all nonlinear filters used for a given number of channels.

```
nl_shifts = np.zeros((channels,channels,2),dtype=int)
for i in range(channels):
    for j in range(channels):
        if i == j:
            nl_shifts[i,i,0] = max_nl_shift - nl_mem
            nl_shifts[i,i,1] = max_nl_shift - nl_mem
        elif i > j:
            nl_shifts[i,j,1] = max_nl_shift - nl_mem_nc[i-j,1]
            nl_shifts[i,j,0] = max_nl_shift - nl_mem_nc[i-j,0]
        else:
            nl_shifts[i,j,1] = max_nl_shift - nl_mem_nc[j-i,0]
            nl_shifts[i,j,0] = max_nl_shift - nl_mem_nc[j-i,1]
```

Since all nonlinear filters (SPM and XPM) in general have different width and different numbers of left and right symbols are used, before applying these filters the input arrays must be properly processed. The resulting arrays should be the same length and the corresponding symbols on different spectral channels should be opposite each other. Therefore, by analogy with linear convolution filters, array `nl_shifts`, containing for each channel pair the number of first and last symbols that should be discarded before applying the corresponding filter, is declared. This array is filled following the same reasoning as for arrays `step_shifts` and `last_shifts`. Thus, `nl_shifts[i,j,0]` contains the number of first symbols that should be discarded before applying XPM filter (or SPM filter if `i = j`) that takes into account the nonlinear effect of channel `j` on channel `i`. `nl_shifts[i,j,1]` is the same, but for the last symbols. After applying all nonlinear convolution filters, an array corresponding to the desired NN output is reduced by `max_nl_shift` symbols on each side by analogy with arrays `step_output_shifts` and `last_output_shifts`.

```
full_delay = num_step * (filt_CDC_delay + max_nl_shift)
if last_step_length > 0:
    full_delay += filt_CDC_last_delay
if FD_step_length > 0:
    full_delay += filt_FD_delay
```

As mentioned above, after applying the convolution filter, the size of the data array decreases by a certain number of symbols on each side. Variable `full_delay` determines the number of symbols by which the data size on each side will decrease after num_step of linear CDC and nonlinear activation function layers, last CDC and FD filters, if `last_step_length` > 0 and `FD_step_length` > 0.

5 Function

Define all support functions.

5.1 BER and CD

Functions for BER calculating and CD compensation.

```
def symbols_to_codes(data_complex):
    codes = np.zeros(len(data_complex), dtype=np.int32)
    for i in range(len(data_complex)):
        codes[i] = np.argmin(abs(gray_symbols_16qam - data_complex[i]))

    return codes
```

This function for the input array of complex symbols `data_complex` gives an array of numbers of constellation diagram symbols (`codes`), the distance to which is minimal. 16-QAM symbols `gray_symbols_16qam` is defined in section 2.

```
def ber_by_codes(tx, rx):
    diff = tx ^ rx
    errors = 0
    for error in diff:
        while error:
            error &= error - 1
            errors +=1

    return 0.25 * errors / len(diff)
```

This function calculates bit-error rate for two sequences of symbols codes `tx` and `rx`.

```
def calculate_ber(tx, rx):
    return ber_by_codes(symbols_to_codes(tx[0,0,:] + 1j * tx[0,1,:]),
        symbols_to_codes(rx[0,0,:] + 1j * rx[0,1,:]))
```

This function calculates BER for two symbols arrays using support functions. Arrays should have a size of 1x2xN, which corresponds to the real and imaginary parts of N complex symbols.

```
def demapper(tx, rx):
    phi = (1+np.sqrt(5))/2
    xl = 0.9
    xr = 1.1
    for j in range(10):
        x1 = xr - (xr-xl)/phi
        x2 = xl + (xr-xl)/phi
        y1 = calculate_ber(tx, rx * x1)
        y2 = calculate_ber(tx, rx * x2)
        if y1 >= y2:
            xl = x1
        else:
            xr = x2
        x = (x1+x2)/2

    return x
```

This function for two arrays of symbols `tx` and `rx` returns a constant `x`, which minimizes BER between `tx` and `rx * x`. To find the minimum, the golden-section search is used, and 10 iterations of this method are performed.

```
def cd_operator(data, size, length, chFreq, direction):
    dw = 2*math.pi*Fsym/size
    w = np.arange(-size/2, size/2,1) * dw
    w = np.fft.fftshift(w)

    if direction == 'f':
        s = 1
    if direction == 'b':
        s = -1

    fft_data = np.fft.fft(data)
    fft_data = fft_data * np.exp(s * 1j * b2 / 2 * (w + 2 * math.pi * chFreq) ** 2
        * length)
    dataCD = np.fft.ifft(fft_data)

    return dataCD
```

This function compensates for CD or recovers of the signal dispersion broadening. At the input, the function receives an array `data` of `size` complex symbols, propagation length, frequency at which the signal propagates `chFreq`. The parameter `direction` sets the direction in which the signal propagates. If `direction = 'f'`, then function recovers

of the signal dispersion broadening. `direction = 'b'` corresponds to the case of CD compensation. The function returns the resulting complex array `dataCD` of the same length as the input.

```
1  def create_metadata():
2      metadata1 = dict( (name, eval(name)) for name in ['channels', 'channel_numbers',
3          'train_portion', 'train_data_size', 'CDC_data_size', 'num_step',
4          'filt_CDC_width', 'nl_mem', 'nl_mem_nc1_1', 'nl_mem_nc1_2', 'nl_mem_nc2_1',
5          'nl_mem_nc2_2', 'nl_mem_nc3_1', 'nl_mem_nc3_2'])
6      metadata2 = dict( (name, eval(name)) for name in ['epochs', 'batch_size','lrate',
7          'lrate_CDC', 'manual_shift', 'sym_filt', 'sym_nonlin', 'save_model_best'])
8
9      now = datetime.datetime.now()
10     date_and_time = now.strftime('%Y%m%d_%H%M')
11     dir_path = results_path + date_and_time + '/'
12     makedirs(dir_path)
13
14     f = open(dir_path + 'metadata.md', 'w')
15     f.write(now.strftime('%Y-%m-%d %H:%M') + '\n\n')
16
17     for x, y in metadata1.items():
18         f.write('{}: {}\n'.format(x, y))
19     if manual_shift:
20         f.write('filt_CDC_last_width: {}\n'.format(filt_CDC_last_width))
21         f.write('filt_FD_width: {}\n'.format(filt_FD_width))
22     for x, y in metadata2.items():
23         f.write('{}: {}\n'.format(x, y))
24     f.close()
25
26     return dir_path
```

This function creates a new folder located in `results_path`, the name of which consists of the date and time the script was launched. A file `metadata.md` is created in this folder containing all the parameters of this run (these parameters are listed in lines 2-7). The function returns the path to the created folder.

5.2 Initialization and Dataloader

Custom initialization and dataloader functions.

```
1  class CustomInit(mx.init.Initializer):
2      def __init__(self, filt):
3          super(CustomInit, self).__init__()
4          self.filt = filt
5
```

```
    def _init_weight(self, _, arr):
        arr[:] = self.filt
```

Custom initialization class that initializes the corresponding parameters or layer weights with a given array `filt`.

```
def dataloader(X, y, batch, enableShuffle):
```

This function creates an object of MXNet DataLoader type for data arrays `X` and `y`. This object contains batches, consisting of arrays fed to the NN input (`data`) and arrays with the desired outputs of the neural network (`label`). It also receives as input the desired batch size (`batch`) and parameter `enableShuffle` that determines whether to shuffle the samples.

```
    size = X.shape[1]
    batch = min(batch, size)
```

Variable `size` determines the size of the input array `X` and if it is smaller than the desired batch size, `batch` is taken equal to `size`.

```
    d = size / batch
    if d%1 > 0.2:
        batch = np.floor(size / np.ceil(d)).astype(int)
    batch = batch - batch%2
    number_of_bathes = np.floor(size / batch).astype(int)
```

In this block, the size of the batch is chosen so that it is close to the desired, but at the same time a small number of symbols are discarded when the entire data array is divided into batch of the same size. Scalar `d` determines how many batches the data size can be divided (in general, this is not an integer). If the fractional part of `d` is greater than 0.2, that is, if 0.2·`batch` symbols will not be used, then the batch size is set so that it is maximal, in which at least `d` identical batches fit into the data array. Next, taking into account the new batch size, the number of batches (variable `number_of_bathes`) into which the data arrays will be divided is determined.

```
    data = nd.empty((number_of_bathes, data_dimension, batch))
    if manual_shift:
        label = nd.empty((number_of_bathes, data_dimension, batch-2*full_delay -
            max_full_shift))
    else:
        label = nd.empty((number_of_bathes, data_dimension, batch-2*full_delay))
```

In this code block, variables `data` and `label` into which arrays `X` and `y` will be written after splitting into batches are initialized. The size of one batch for variable `label` is set in such a way as to correspond to the size of the data received at the NN output, after an array of size `batch` was fed to the input (see section 4.2 for more information about reducing the size of array after propagation through a neural network).

```
    for i in range(number_of_bathes):
        data[i,:,:] = nd.array(X[:,i*batch:(i+1)*batch])
        for j in range(channels):
            if manual_shift:
                label[i,j*4:(j+1)*4,:] = nd.array(y[j*4:(j+1)*4,i*batch+full_delay +
                    full_output_shifts[0]:(i+1)*batch-full_delay-full_output_shifts[1]])
            else:
                label[i,j*4:(j+1)*4,:] = nd.array(y[j*4:(j+1)*4,i*batch+full_delay:
                    (i+1)*batch-full_delay])
```

In this block, in a loop for batch numbers, arrays `data` and `label` are filled. The part of `X` array corresponding to `i`th batch is written to the appropriate column (according to the first dimension) of the array `data`. It is also converted to MXNet NDArray format using the command `nd.array()`. Further, in a loop for all channels, part of `y` array, taking into account the data size reduction due to the use of convolution filters (and symbol discarding if `manual_shift = True`), fills the array `label` in the same way.

```
    dataset = gluon.data.dataset.ArrayDataset(data, label)
```

In this line, arrays `data` and `label` are combined in a MXNet Dataset object `dataset`. It is necessary to create an object of MXNet DataLoader type. A DataLoader is used to create mini-batches of samples from arrays of data, and provides a convenient iterator interface for looping these batches.

```
    return gluon.data.DataLoader(dataset, batch_size=1, shuffle=enableShuffle),
        batch, number_of_bathes
```

This functuon returns DataLoader type object obtained from `dataset` with parameter `shuffle` specified by variable `enableShuffle`. It also returns size and number of batches.

5.3 Linear and Nonlinear layers

Custom layers for CD compensation and nonlinear activation functions.

5.3.1 ComplexConvRandom

Custom complex convolution layer for CD compensation with initially random weights.

```
class ComplexConvRandom(gluon.Block):
    def __init__(self, kernel_size, dimension, shift=np.array([[0,0]]), pol=2,
        enable_shift=True, sym=False, strides=1, **kwargs):
        super(ComplexConvRandom, self).__init__(**kwargs)
```

This layer has the following parameters: number of filter coefficients `kernel_size`, input array dimension `dimension`, array `shift` containing how many first and last symbols in each channel should be discarded after applying the filter (if it is not defined by the user, then it is initialized as array of zeros `np.array([[0,0]])`), number of processed polarizations `pol` (by default it is 2), parameter `enable_shift` determine whether symbols is discarded due to time shifts of signals propagating at different frequencies (by default it is `True`), parameter `sym` determines whether this filter should be symmetrical (by default it is `False`). The remaining lines in this block are required by MXNet syntax.

```
        with self.name_scope():
            if len(shift) == 1 and np.sum(shift[0,:]) == 0:
                    shift = np.zeros((int(dimension/(2*pol)), 2),dtype=int)
            self.dimension = dimension
            self.shift = shift
            self.pol = pol
            self.kernel_size = kernel_size
            self.enable_shift = enable_shift
            self.sym = sym
            self.weight_re = self.params.get('weight_re', allow_deferred_init=True,
                shape=(int(dimension/(2*pol)),1,kernel_size))
            self.weight_im = self.params.get('weight_im', allow_deferred_init=True,
                shape=(int(dimension/(2*pol)),1,kernel_size))
```

In this block layer inner variables are declared. If `shift` was not defined by the user, it initialized by zeros for all channels. In the last 4 lines variables containing the weights of the real and imaginary parts of the convolution layers are declared. It is assumed here that the same filter is used for each polarization of one channel, that is, there are `dimension/(2*pol)` (number of spectral channels processed) pairs of filters in total. All weights of declared filters are initialized with random numbers.

```
    def forward(self, z):
        with z.context:
            chan = int(self.dimension/(2*self.pol))
```

Variable `z` corresponds to the input data array. Variable `chan` determines the number of processed channels.

```
            if self.sym:
                filt_re = nd.concat(self.weight_re.data(), nd.flip(self.weight_re.data(),
                    axis=2)[:,:,1:], dim=2)
                filt_im = nd.concat(self.weight_im.data(), nd.flip(self.weight_im.data(),
                    axis=2)[:,:,1:], dim=2)
                kernel = 2 * self.kernel_size - 1
            else:
                filt_re = self.weight_re.data()
                filt_im = self.weight_im.data()
                kernel = self.kernel_size
```

If the filters should be symmetrical, variables `self.weight_re` and `self.weight_im` contain only unknown coefficients corresponding to the left side and the central coefficient of the filter. For the convenience of using the convolution function, the left side of the filter is reversed (function `nd.flip()`) and attached as the right side (function `nd.concat()`), thereby forming an array of coefficients (`filt_re` for the real part and `filt_im` for the imaginary). If `self.sym = False` filter coefficients are suitable and they just fill arrays `filt_re` and `filt_im`.

```
            y = nd.Convolution(data = z[:,0::2*self.pol,:], weight = filt_re,
                no_bias=True, num_filter=chan, kernel=kernel, num_group=chan) -
                nd.Convolution(data = z[:,1::2*self.pol,:], weight = filt_im,
                no_bias=True, num_filter=chan, kernel=kernel, num_group=chan)

            y = nd.concat(y, nd.Convolution(data = z[:,0::2*self.pol,:],
                weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
                num_group=chan) + nd.Convolution(data = z[:,1::2*self.pol,:],
                weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
                num_group=chan), dim=1)
```

As noted above (??) single complex-valued convolution can be realised using four real-valued convolutions and two addition operations. Since an individual filter is used for each channel, we can perform a convolution operation for real or imaginary part pf one polarization for all channels simultaneously using the function `nd.Convolution()`. It has the following parameters: input array `data`, convolution filter coefficient array `weight`, parameter `no_bias` determines whether bias is used (by default it is `True`), parameter `num_filter` sets the number of concurrently performed convolutions (the number of convolution filters that is equal to the number of processed channels), number of filter coefficients `kernel_size`, and parameter `num_group` determines the number of

independent processed data groups, it means that the results of convolutions from different channels in the output array will be in different columns and will not be mixed together. For example, in lines 1 and 2, there is a convolution of real part of the first polarization for all channels (for more information about column orders in the data array, see section 6). In general, this block corresponds to the calculation of complex convolutions for the first polarization of each channel in accordance with the formula (??).

```
1             for i in range(2, 2*self.pol, 2):
2                 y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
3                     weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
4                     num_group=chan) - nd.Convolution(data = z[:,i+1::2*self.pol,:],
5                     weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
6                     num_group=chan), dim=1)
7                 y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
8                     weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
9                     num_group=chan) + nd.Convolution(data = z[:,i+1::2*self.pol,:],
10                    weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
11                    num_group=chan), dim=1)
12
13            for i in range(2, 2*self.pol, 2):
14                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
15                    weight = filt_re, no_bias=True, num_filter=chan,
16                    kernel=self.kernel_size, num_group=chan) - nd.Convolution(data =
17                    z[:,i+1::2*self.pol,:], weight = filt_im, no_bias=True,
18                    num_filter=chan, kernel=self.kernel_size, num_group=chan), dim=1)
19                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
20                    weight = filt_im, no_bias=True, num_filter=chan,
21                    kernel=self.kernel_size, num_group=chan) + nd.Convolution(data =
22                    z[:,i+1::2*self.pol,:], weight = filt_re, no_bias=True,
23                    num_filter=chan, kernel=self.kernel_size, num_group=chan), dim=1)
```

This block corresponds to the calculation of complex convolutions for the second polarization (if `pol = 2`, otherwise it will be skipped) of each channel, by analogy with the previous block.

```
1             v = nd.slice(y, begin=(None, 0, None), end=(None, self.dimension, None),
2                 step=(1, chan, 1))
3             if self.enable_shift:
4                 v = v[:,:,self.shift[0,0]:v.shape[2]-self.shift[0,1]]
```

If parameter `self.enable_shift` is set as `True` the corresponding number of first and last symbols should be discarded. To do this, first, all the columns corresponding to the first channel (including real and imaginary part and polarizations) are sliced in lines 1 and

2. Next the number of symbols corresponding to the first channel is discarded from the beginning (`self.shift[0,0]`) and end of the resulting array (`self.shift[0,1]`).

```
            for i in range(1, chan):
                u = nd.slice(y, begin=(None, i, None), end=(None, self.dimension, None),
                    step=(1, chan, 1))
                if self.enable_shift:
                    u = u[:,:,self.shift[i,0]:u.shape[2]-self.shift[i,1]]
                v = nd.concat(v, u, dim=1)

            return v
```

Further, the same procedure is performed for the remaining channels. All resulting arrays are combined in array `v`. And `v` is returned as the output of this layer.

5.3.2 ComplexConvPredefined

Custom complex convolution layer for CD compensation with predefined weights.

```
class ComplexConvPredefined(gluon.Block):
    def __init__(self, weightsRe, weightsIm, kernel_size, dimension,
        shift=np.array([[0,0]]), pol=2, enable_shift=True, sym=False,
            strides=1, **kwargs):
        super(ComplexConvPredefined, self).__init__(**kwargs)
        with self.name_scope():
            if len(shift) == 1 and np.sum(shift[0,:]) == 0:
                    shift = np.zeros((int(dimension/(2*pol)), 2),dtype=int)
            self.dimension = dimension
            self.shift = shift
            self.pol = pol
            self.kernel_size = kernel_size
            self.enable_shift = enable_shift
            self.sym = sym
            self.weight_re = self.params.get('weight_re', allow_deferred_init=True,
                shape=(int(dimension/(2*pol)),1,kernel_size), init=CustomInit(weightsRe))
            self.weight_im = self.params.get('weight_im', allow_deferred_init=True,
                shape=(int(dimension/(2*pol)),1,kernel_size), init=CustomInit(weightsIm))

    def forward(self, z):
        with z.context:
            chan = int(self.dimension/(2*self.pol))

            if self.sym:
                filt_re = nd.concat(self.weight_re.data(), nd.flip(self.weight_re.data(),
                    axis=2)[:,:,1:], dim=2)
                filt_im = nd.concat(self.weight_im.data(), nd.flip(self.weight_im.data(),
                    axis=2)[:,:,1:], dim=2)
```

```
                kernel = 2 * self.kernel_size - 1
            else:
                filt_re = self.weight_re.data()
                filt_im = self.weight_im.data()
                kernel = self.kernel_size

            y = nd.Convolution(data = z[:,0::2*self.pol,:], weight = filt_re,
                no_bias=True, num_filter=chan, kernel=kernel, num_group=chan) -
                nd.Convolution(data = z[:,1::2*self.pol,:], weight = filt_im,
                no_bias=True, num_filter=chan, kernel=kernel, num_group=chan)

            y = nd.concat(y, nd.Convolution(data = z[:,0::2*self.pol,:],
                weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
                num_group=chan) + nd.Convolution(data = z[:,1::2*self.pol,:],
                weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
                num_group=chan), dim=1)

            for i in range(2, 2*self.pol, 2):
                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
                    weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
                    num_group=chan) - nd.Convolution(data = z[:,i+1::2*self.pol,:],
                    weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
                    num_group=chan), dim=1)
                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
                    weight = filt_im, no_bias=True, num_filter=chan, kernel=kernel,
                    num_group=chan) + nd.Convolution(data = z[:,i+1::2*self.pol,:],
                    weight = filt_re, no_bias=True, num_filter=chan, kernel=kernel,
                    num_group=chan), dim=1)

            for i in range(2, 2*self.pol, 2):
                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
                    weight = filt_re, no_bias=True, num_filter=chan,
                    kernel=self.kernel_size, num_group=chan) - nd.Convolution(data =
                    z[:,i+1::2*self.pol,:], weight = filt_im, no_bias=True,
                    num_filter=chan, kernel=self.kernel_size, num_group=chan), dim=1)
                y = nd.concat(y, nd.Convolution(data = z[:,i::2*self.pol,:],
                    weight = filt_im, no_bias=True, num_filter=chan,
                    kernel=self.kernel_size, num_group=chan) + nd.Convolution(data =
                    z[:,i+1::2*self.pol,:], weight = filt_re, no_bias=True,
                    num_filter=chan, kernel=self.kernel_size, num_group=chan), dim=1)

            v = nd.slice(y, begin=(None, 0, None), end=(None, self.dimension, None),
                step=(1, chan, 1))
            if self.enable_shift:
                v = v[:,:,self.shift[0,0]:v.shape[2]-self.shift[0,1]]
            for i in range(1, chan):
                u = nd.slice(y, begin=(None, i, None), end=(None, self.dimension, None),
                    step=(1, chan, 1))
                if self.enable_shift:
                    u = u[:,:,self.shift[i,0]:u.shape[2]-self.shift[i,1]]
```

```
79                  v = nd.concat(v, u, dim=1)
80
81              return v
```

This layer is the same as the `ComplexConvRandom`, except that filter weights are initialized by arrays `weightsRe` and `weightsIm` (`init=CustomInit(weightsRe)` and `init=CustomInit(weightsIm)` in lines 16 and 18).

5.3.3 KerrActivationEnhanced_1ch

Layer corresponding to the nonlinear enhanced activation function for 1 channel.

```
1  class KerrActivationEnhanced_1ch(gluon.Block):
2      def __init__(self, dimension, tensor_intra, nl_shifts, nl_coef=0.1,
3          strides=1, **kwargs):
4          super(KerrActivationEnhanced_1ch, self).__init__()
```

This layer has the following parameters: input array dimension `dimension`, array `tensor_intra` containing predefined weights for SPM filter, array `nl_shift` containing for each channel pair the number of first and last symbols that should be discarded before applying the filter, parameter `nl_coef` with the initial approximation of the nonlinear parameter γ (by default it is 0.1). The remaining lines in this block are required by MXNet syntax.

```
1          chan = 1
2          self.chan = chan
3          self.nl_shifts = nl_shifts
4          with self.name_scope():
5              self.conv_intra = self.params.get('conv_intra', allow_deferred_init=True,
6                  shape=(chan,1,memory_size), init=CustomInit(tensor_intra))
7              self.gamma = self.params.get('gamma', shape=(1,), allow_deferred_init=True,
8                  init=mx.init.Constant(nl_coef))
```

In this block layer inner variables are declared. Variable `chan` is equal to 1, since this layer is for one processed channel. Variable `self.conv_intra` contains the weights for the nonlinear SPM filter for the channel in consideration. It is initialized by `tensor_intra` and the filter width is memory_size. Parameter `self.gamma` corresponds to the nonlinear parameter γ and its initial value is equal to `nl_coef`.

```
    def forward(self, z):
        power = nd.square(z[:,0:1,:]) + nd.square(z[:,1:2,:]) + nd.square(z[:,2:3,:]) +
            nd.square(z[:,3:4,:])
        power_intra = power[:,:,self.nl_shifts[0,0,0]:power.shape[2] -
            self.nl_shifts[0,0,1]]
```

In this block, the power of the processed channel (`power`) is first calculated as the sum of the squares of the real and imaginary parts of both polarizations. Next, from the beginning and from the end of the array, the necessary number of symbols is discarded for the further use of SPM filter.

```
        if sym_nonlin:
            tens_intra = nd.concat(self.conv_intra.data(),
                nd.flip(self.conv_intra.data(), axis=2)[:,:,1:], dim=2)
            power = nd.Convolution(data = power_intra, weight = tens_intra, no_bias=True,
                num_filter=self.chan, kernel=2*memory_size-1, num_group=self.chan)
        else:
            power = nd.Convolution(data = power_intra, weight = self.conv_intra.data(),
                no_bias=True, num_filter=self.chan, kernel=memory_size,
                num_group=self.chan)
```

If SPM filter should be symmetric, array of unknown filter coefficients from `self.conv_intra` is transformed to a new array `tens_intra` by analogs with how it was done for linear filters in section 5.3.1. Next, the real convolution of the `power` array with the SPM filter coefficients is performed. It corresponds to the use of the enhanced SSFM at the nonlinear step of DBP (see ??). Since this layer is for one channel, only one filter is involved in the convolution.

```
        u = nd.cos(self.gamma.data() * power) *
            z[:,0,max_nl_shift:z.shape[2]-max_nl_shift] +
            nd.sin(self.gamma.data() * power) *
            z[:,1,max_nl_shift:z.shape[2]-max_nl_shift]
        u = nd.concat(u, nd.cos(self.gamma.data() * power) *
            z[:,1,max_nl_shift:z.shape[2]-max_nl_shift] -
            nd.sin(self.gamma.data() * power) *
            z[:,0,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
        u = nd.concat(u, nd.cos(self.gamma.data() * power) *
            z[:,2,max_nl_shift:z.shape[2]-max_nl_shift] +
            nd.sin(self.gamma.data() * power) *
            z[:,3,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
        u = nd.concat(u, nd.cos(self.gamma.data() * power) *
            z[:,3,max_nl_shift:z.shape[2]-max_nl_shift] -
            nd.sin(self.gamma.data() * power) *
            z[:,2,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)

```

```
18          return u
```

In this block of code, the complex nonlinear activation function for one channels is calculated, as described in the formula ??. Array `z` is reduced appropriately to match the size of the power array after discarding the symbols and applying convolution. Array `u` contains the resulting arrays for the real and imaginary parts of the two polarizations of the processed channel. And it is returned as the output of this layer.

5.3.4 KerrActivationEnhanced_2ch

Layer corresponding to the nonlinear enhanced activation function for 2 channels.

```
1  class KerrActivationEnhanced_2ch(gluon.Block):
2      def __init__(self, dimension, tensor_intra, tensor_inter, nl_shifts, nl_coef=0.1,
3          strides=1, **kwargs):
4          super(KerrActivationEnhanced_2ch, self).__init__()
5          chan = int(dimension/4)
6          self.chan = chan
7          self.nl_shifts = nl_shifts
8          with self.name_scope():
9              self.conv_intra = self.params.get('conv_intra', allow_deferred_init=True,
10                 shape=(chan,1,memory_size), init=CustomInit(tensor_intra))
11             self.conv_inter = gluon.nn.Conv1D(channels=2, in_channels=2, groups=2,
12                 kernel_size=memory_size_nc1, strides=strides, activation=None,
13                 use_bias=False, weight_initializer=CustomInit(tensor_inter))
14             self.gamma = self.params.get('gamma', shape=(1,), allow_deferred_init=True,
15                 init=mx.init.Constant(nl_coef))
```

This layer is similar to the previous one with the exception of some points. For example, here, variable `self.conv_inter` corresponding to nonlinear XPM filters for the closest adjacent spectral channels is additionally declared. It contains two real filters with `memory_size_nc1` coefficients each, the firsts takes into account the nonlinear effect of channel 1 on channel 2, and the second takes into account the nonlinear effect of channel 2 on channel 1.

```
1      def forward(self, z):
2          power = nd.square(z[:,0:1,:]) +  nd.square(z[:,1:2,:]) + nd.square(z[:,2:3,:]) +
3              nd.square(z[:,3:4,:])
4          power_intra = power[:,:,self.nl_shifts[0,0,0]:power.shape[2] -
5              self.nl_shifts[0,0,1]]
6          power_inter = power[:,:,self.nl_shifts[0,1,0]:power.shape[2] -
7              self.nl_shifts[0,1,1]]
```

In this block, a variable, which is used to take into account the influence of the first processed channel on the second, is additionally introduced.

```
        power = nd.square(z[:,4:5,:]) +  nd.square(z[:,5:6,:]) + nd.square(z[:,6:7,:]) +
            nd.square(z[:,7:8,:])
        power_intra = nd.concat(power_intra,
            power[:,:,self.nl_shifts[1,1,0]:power.shape[2]-self.nl_shifts[1,1,1]], dim=1)
        power_inter = nd.concat(power_inter,
            power[:,:,self.nl_shifts[1,0,0]:power.shape[2]-self.nl_shifts[1,0,1]], dim=1)
```

In this code of block, the power of the second channel is first calculated. Then, after discarding the corresponding number of symbols, the part of `power` that will be used to account the intra-channel interaction for the second channel is added to the array `power_intra`, and the part of `power` that will be used to account for the impact of the second channel on the first is added to the array `power_inter`.

```
        if sym_nonlin:
            tens_intra = nd.concat(self.conv_intra.data(),
                nd.flip(self.conv_intra.data(), axis=2)[:,:,1:], dim=2)
            power_intra = nd.Convolution(data = power_intra, weight = tens_intra,
                no_bias=True, num_filter=self.chan, kernel=2*memory_size-1,
                num_group=self.chan)
        else:
            power_intra = nd.Convolution(data = power_intra,
                weight = self.conv_intra.data(), no_bias=True, num_filter=self.chan,
                kernel=memory_size, num_group=self.chan)
```

In this block, by analogy with the layer for one channel, a real convolution corresponding to the SPM filter is performed, except that both channels are processed simultaneously and independently (`num_filter=self.chan` and `num_group=self.chan`).

```
        power_inter = self.conv_inter(power_inter)
```

Here, the convolution for XPM-1 filters is similarly performed for both channels.

```
        power = power_intra + nd.concat(power_inter[:,1:2,:], power_inter[:,0:1,:])
```

In this line, array `power` corresponding to the enhanced SSFM method for 2 channels is calculated according to the formula ??.

```
        u = nd.cos(self.gamma.data() * power[:,0:1,:]) *
            z[:,0,max_nl_shift:z.shape[2]-max_nl_shift] + nd.sin(self.gamma.data() *
            power[:,0:1,:]) * z[:,1,max_nl_shift:z.shape[2]-max_nl_shift]
        u = nd.concat(u, nd.cos(self.gamma.data() * power[:,0:1,:]) *
            z[:,1,max_nl_shift:z.shape[2]-max_nl_shift] - nd.sin(self.gamma.data() *
            power[:,0:1,:]) * z[:,0,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
        for i in range(1, self.chan * 2):
            ind = int(i / 2)
            u = nd.concat(u, nd.cos(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i,max_nl_shift:z.shape[2]-max_nl_shift] +
                nd.sin(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i+1,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
            u = nd.concat(u, nd.cos(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i+1,max_nl_shift:z.shape[2]-max_nl_shift] -
                nd.sin(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)

        return u
```

In this block of code, the complex nonlinear activation function for both channels is calculated, by analogy with the layer for one channel. Array `u` containing the resulting arrays for the real and imaginary parts of the two polarizations of the processed channels is returned as the output of this layer.

5.3.5 KerrActivationEnhanced_4ch

Layer corresponding to the nonlinear enhanced activation function for 2 channels.

```
class KerrActivationEnhanced_4ch(gluon.Block):
    def __init__(self, dimension, tensor_intra, tensor_inter1, tensor_inter2,
        tensor_inter3, nl_shifts, nl_coef=0.1, strides=1, **kwargs):
        super(KerrActivationEnhanced_4ch, self).__init__()
        chan = int(dimension/4)
        self.chan = chan
        self.nl_shifts = nl_shifts
        with self.name_scope():
            self.conv_intra = self.params.get('conv_intra', allow_deferred_init=True,
                shape=(chan,1,memory_size), init=CustomInit(tensor_intra))
            self.conv_inter1 = gluon.nn.Conv1D(channels=6, in_channels=6, groups=6,
                kernel_size=memory_size_nc1, strides=strides, activation=None,
                use_bias=False, weight_initializer=CustomInit(tensor_inter1))
            self.conv_inter2 = gluon.nn.Conv1D(channels=4, in_channels=4, groups=4,
                kernel_size=memory_size_nc2, strides=strides, activation=None,
                use_bias=False, weight_initializer=CustomInit(tensor_inter2))
            self.conv_inter3 = gluon.nn.Conv1D(channels=2, in_channels=2, groups=2,
                kernel_size=memory_size_nc3, strides=strides, activation=None,
                use_bias=False, weight_initializer=CustomInit(tensor_inter3))
```

```
            self.gamma = self.params.get('gamma', shape=(1,), allow_deferred_init=True,
                init=mx.init.Constant(nl_coef))

    def forward(self, z):
        power = nd.square(z[:,0:1,:]) +  nd.square(z[:,1:2,:]) + nd.square(z[:,2:3,:]) +
            nd.square(z[:,3:4,:])
        power_intra = power[:,:,self.nl_shifts[0,0,0]:power.shape[2] -
            self.nl_shifts[0,0,1]]
        power_inter1 = power[:,:,self.nl_shifts[0,1,0]:power.shape[2] -
            self.nl_shifts[0,1,1]]
        power_inter2 = power[:,:,self.nl_shifts[0,2,0]:power.shape[2] -
            self.nl_shifts[0,2,1]]
        power_inter3 = power[:,:,self.nl_shifts[0,3,0]:power.shape[2] -
            self.nl_shifts[0,3,1]]

        for i in range(1, self.chan):
            power = nd.square(z[:,4*i:4*i+1,:]) +  nd.square(z[:,4*i+1:4*i+2,:]) +
                nd.square(z[:,4*i+2:4*i+3,:]) +  nd.square(z[:,4*i+3:4*i+4,:])
            for j in range(self.chan):
                if i == j:
                    power_intra = nd.concat(power_intra,
                        power[:,:,self.nl_shifts[i,i,0]:power.shape[2] -
                        self.nl_shifts[i,i,1]], dim=1)
                elif abs(i-j) == 1:
                    power_inter1 = nd.concat(power_inter1,
                        power[:,:,self.nl_shifts[i,j,0]:power.shape[2] -
                        self.nl_shifts[i,j,1]], dim=1)
                elif abs(i-j) == 2:
                    power_inter2 = nd.concat(power_inter2,
                        power[:,:,self.nl_shifts[i,j,0]:power.shape[2] -
                        self.nl_shifts[i,j,1]], dim=1)
                else:
                    power_inter3 = nd.concat(power_inter3,
                    power[:,:,self.nl_shifts[i,j,0]:power.shape[2] -
                    self.nl_shifts[i,j,1]], dim=1)

        if sym_nonlin:
            tens_intra = nd.concat(self.conv_intra.data(),
                nd.flip(self.conv_intra.data(), axis=2)[:,:,1:], dim=2)
            power_intra = nd.Convolution(data = power_intra, weight = tens_intra,
                no_bias=True, num_filter=self.chan, kernel=2*memory_size-1,
                num_group=self.chan)
        else:
            power_intra = nd.Convolution(data = power_intra,
                weight = self.conv_intra.data(), no_bias=True, num_filter=self.chan,
                kernel=memory_size, num_group=self.chan)
        power_inter1 = self.conv_inter1(power_inter1)
        power_inter2 = self.conv_inter2(power_inter2)
        power_inter3 = self.conv_inter3(power_inter3)
        power_inter = [power_inter1, power_inter2, power_inter3]
```

```
        power = power_intra
        power = power + nd.concat(power_inter1[:,1:2,:], power_inter1[:,0:1,:],
            power_inter2[:,0:1,:], power_inter3[:,0:1,:])
        power = power + nd.concat(power_inter2[:,2:3,:], power_inter1[:,3:4,:],
            power_inter1[:,2:3,:], power_inter2[:,1:2,:])
        power = power + nd.concat(power_inter3[:,1:2,:], power_inter2[:,3:4,:],
            power_inter1[:,5:6,:], power_inter1[:,4:5,:])

        u = nd.cos(self.gamma.data() * power[:,0:1,:]) *
            z[:,0,max_nl_shift:z.shape[2]-max_nl_shift] + nd.sin(self.gamma.data() *
            power[:,0:1,:]) * z[:,1,max_nl_shift:z.shape[2]-max_nl_shift]
        u = nd.concat(u, nd.cos(self.gamma.data() * power[:,0:1,:]) *
            z[:,1,max_nl_shift:z.shape[2]-max_nl_shift] - nd.sin(self.gamma.data() *
            power[:,0:1,:]) * z[:,0,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
        for i in range(1, self.chan * 2):
            ind = int(i / 2)
            u = nd.concat(u, nd.cos(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i,max_nl_shift:z.shape[2]-max_nl_shift] +
                nd.sin(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i+1,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)
            u = nd.concat(u, nd.cos(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i+1,max_nl_shift:z.shape[2]-max_nl_shift] -
                nd.sin(self.gamma.data() * power[:,ind:ind+1,:]) *
                z[:,2*i,max_nl_shift:z.shape[2]-max_nl_shift], dim=1)

        return u
```

This layer is the same as the previous two layers except that two additional nonlinear filters (XPM-2 and XPM-3) are used here.

5.3.6 ComplexConvSeq

This function allows you to create structures for nested neural networks using MXNet syntax. Each nested neural network is part of the external NN.

```
class ComplexConvSeq(nn.Sequential):
    def __init__(self, chan, **kwargs):
        super(ComplexConvSeq, self).__init__(**kwargs)
        self.chan = chan
    def forward(self, z):
        v = self._children['0'](z)
        u = v
        for i in range(1, num_step):
            u = self._children[str(i)](u)
            if manual_shift:
                v = nd.concat(v[:,:,step_output_shifts[0]+filt_CDC_delay:v.shape[2] -
                    filt_CDC_delay-step_output_shifts[1]], u, dim=0)
```

```
            else:
                v = nd.concat(v[:,:,filt_CDC_delay:v.shape[2]-filt_CDC_delay], u, dim=0)
        if last_step_length > 0:
            u = self._children[str(num_step)](u)
            if manual_shift:
                v = nd.concat(v[:,:,last_output_shifts[0]+filt_CDC_last_delay:
                    v.shape[2]-filt_CDC_last_delay-last_output_shifts[1]], u, dim=0)
            else:
                v = nd.concat(v[:,:,filt_CDC_last_delay:v.shape[2]-filt_CDC_last_delay],
                    u, dim=0)
        if FD_step_length > 0:
            u = self._children[str(len(self._children.items())-1)](u)
            v = nd.concat(v[:,:,filt_FD_delay:v.shape[2]-filt_FD_delay], u, dim=0)
        return v
```

This structure is needed for joint optimization of CDC filters. When using it, the training of the entire neural network is organized so that not only the full sequence of filters compensates for the all accumulated dispersion, but also that the first CDC filter compensates for dispersion for a distance of `step_length`, the first two CDC filters compensate CD for a distance of 2·`step_length`, the first three CDC filters – for a distance of 3·`step_length` and so on. Here, each such internal sequence of filters is considered as a separate neural network, thus obtaining a sequence of neural networks nested into each other. Thus, such a structure has as output not only a resulting array after data propagation through the entire neural network, but also arrays obtained after each of the filters. In this code of block, array `v` stores the output data for all nested neural networks and after each new layer its size is reduced by the corresponding value (due to convolution operations and symbol discarding) and a new NN output is added to it.

5.4 CDC and FD filters

Find CDC and FD filters coefficients and joint optimization of the filters.

5.4.1 get_CDCfilt_step

Function for finding the coefficients of complex filters that compensates for CD for all processed channels.

```
def get_CDCfilt_step(data_real, width, length, shift, output_shift):
```

This function has the following parameters: array `data_real` consisting of columns with real and imaginary parts of one polarization of one channel, number of filter coefficients `width`, distance `length` for which this filter should compensate for dispersion, array `shift` containing how many first and last symbols in each channel should be discarded after applying the filter, array `output_shift` contains the number of first and last symbols that should be discarded in desired NN output data (for more information see section 4.2).

```
    if sym_filt:
        delay = width - 1
    else:
        delay = int((width-1)/2)
```

Here, depending on whether the filter should be symmetric or not, the number of symbol discarded after the convolution operation is determined.

```
    data = data_real[0,:] + 1j * data_real[1,:]
    size = len(data)
```

In this block, complex array `data` is formed using the columns of array `data_real`. Variable `size` determines the size of this array.

```
    filt_Re = nd.empty((channels, 1, width), dtype=real_t)
    filt_Im = nd.empty((channels, 1, width), dtype=real_t)
```

In these lines, arrays for the real and imaginary parts of the filter coefficients for all processed channels are initialized.

```
    for i in range(channels):
        data_CDC = cd_operator(data, size, length, Fn[channel_numbers[i]], 'b')
        if manual_shift:
            data_CDC = data_CDC[delay+output_shift[0]:size-delay-output_shift[1]]
        else:
            data_CDC = data_CDC[delay:size-delay]
```

For each spectral channel, filter coefficients are found independently. Therefore, in a loop for all processed channels, first, array `data_CDC` is calculated from `data` by CD compensation for a distance `length` and at a frequency `Fn[channel_numbers[i]]`. Next, the size of this array is reduced accordingly due to convolution operations and symbol discarding (if `manual_shift = True`). `data_CDC` will be further used as the desired NN output.

```
        net = gluon.nn.Sequential()
        with net.name_scope():
            net.add(ComplexConvRandom(kernel_size=width, dimension=2,
                shift=shift[i:i+1,:], pol=1, enable_shift=manual_shift, sym=sym_filt))
```

In this block, neural network model `net` with only one complex convolution layer `ComplexConvRandom` is initialized. Since at this stage we find filter coefficients for only one polarization and one channel, we use here `dimension=2` and `pol=1`.

```
        net.initialize(mx.init.Normal(sigma=0.05), ctx=ctx)
        trainer = gluon.Trainer(net.collect_params(), 'Adam',
            {'learning_rate': lrate_CDC})
        mse = gluon.loss.L2Loss()
```

Here, all the weights of the neural network are initialized with random numbers from normal distribution with `sigma=0.05`. NN weights are located on the device defined in `ctx`. As an optimization algorithm `trainer`, Adam optimizer with standard parameters and a learning rate of `lrate_CDC` is used. As a loss function `mse`, the mean squared error is used.

```
        features = nd.empty((1, 2, size))
        features[:,0,:] = nd.array(np.real(data))
        features[:,1,:] = nd.array(np.imag(data))
        features = features.as_in_context(ctx)
        if manual_shift:
            labels = nd.empty((1,2,size-2*delay-abs(np.sum(output_shift))))
        else:
            labels = nd.empty((1,2,size-2*delay))
        labels[:,0,:] = nd.array(np.real(data_CDC))
        labels[:,1,:] = nd.array(np.imag(data_CDC))
        labels = labels.as_in_context(ctx)
```

In this code of block, arrays corresponding to the NN input data (`features`) and the desired NN output (`labels`) are initialized and filled. These arrays are also transferred to the device `ctx`.

```
        lossCond = True
        epoch = 0
        prev_loss = 100
        while lossCond and epoch < 10000 and prev_loss > 1e-4:
            epoch += 1
            tic = time.time()
            train_loss = nd.zeros(1, ctx=ctx)
            with autograd.record():
```

```
                output = net(features)
                loss = mse(output, labels)
            loss.backward()
            trainer.step(size)
            train_loss += loss.mean().asscalar()
            loss.wait_to_read()
            if epoch % 100 == 0:
                print('CDC filter step -- channel', channel_numbers[i], 'epoch', epoch,
                    '-- loss', train_loss.asscalar(), '-- time', time.time()-tic)
                lossCond = abs(train_loss.asscalar() - prev_loss) / prev_loss > 1e-6
                prev_loss = train_loss.asscalar()
```

In this block, the neural network `net` is trained. Training process takes place until the 10000th epoch, until the losses are less than 1e-4 or until losses cease to decrease after each 100 epoch by more than 1e-6 (line 4). Every 100 epoch information about the epoch number, loss and runtime of this epoch is printed. For more information about training commands if this block see section 9.

```
        params = net.collect_params()
        filt_Re[i,:,:] = params[list(params)[0]].data().reshape(width,).asnumpy()
        filt_Im[i,:,:] = params[list(params)[1]].data().reshape(width,).asnumpy()

    return [filt_Re, filt_Im]
```

In this block, the coefficients obtained after training the neural network `net` fill the columns of arrays `filt_Re` and `filt_Im` corresponding to the current channel (command `net.collect_params()` allows to get all weights of the model). Finally, this function returns list of arrays `filt_Re` and `filt_Im`.

5.4.2 get_FDfilt

Function for finding the coefficients of complex FD filters for all processed channels.

```
def get_FDfilt(data_real, width, length):
    delay = int((width-1)/2)
    data = data_real[0,:] + 1j * data_real[1,:]
    size = len(data)
    filtFD_Re = nd.empty((channels, 1, width), dtype=real_t)
    filtFD_Im = nd.empty((channels, 1, width), dtype=real_t)

    for i in range(channels):
        data_CDC = cd_operator(data, size, length, Fn[channel_numbers[i]], 'b')
        data_CDC = data_CDC[delay:size-delay]

```

```
        net = gluon.nn.Sequential()
        with net.name_scope():
            net.add(ComplexConvRandom(kernel_size=width, dimension=2, pol=1,
                enable_shift=False, sym=False))

        net.initialize(mx.init.Normal(sigma=0.05), ctx=ctx)
        trainer = gluon.Trainer(net.collect_params(), 'Adam',
            {'learning_rate': lrate_CDC})
        mse = gluon.loss.L2Loss()

        features = nd.empty((1, 2, size))
        features[:,0,:] = nd.array(np.real(data))
        features[:,1,:] = nd.array(np.imag(data))
        features = features.as_in_context(ctx)
        labels = nd.empty((1,2,size-2*delay))
        labels[:,0,:] = nd.array(np.real(data_CDC))
        labels[:,1,:] = nd.array(np.imag(data_CDC))
        labels = labels.as_in_context(ctx)

        lossCond = True
        epoch = 0
        prev_loss = 100
        while lossCond and epoch < 10000 and prev_loss > 1e-5:
            epoch += 1
            tic = time.time()
            train_loss = nd.zeros(1, ctx=ctx)
            with autograd.record():
                output = net(features)
                loss = mse(output, labels)
            loss.backward()
            trainer.step(size)
            train_loss += loss.mean().asscalar()
            loss.wait_to_read()
            if epoch % 100 == 0:
                print('FD filter -- channel', channel_numbers[i], 'epoch', epoch,
                    '-- loss', train_loss.asscalar(), '-- time', time.time()-tic)
                lossCond = abs(train_loss.asscalar() - prev_loss) / prev_loss > 1e-6
                prev_loss = train_loss.asscalar()

        params = net.collect_params()
        filtFD_Re[i,:,:] = params[list(params)[0]].data().reshape(width,).asnumpy()
        filtFD_Im[i,:,:] = params[list(params)[1]].data().reshape(width,).asnumpy()

    return [filtFD_Re, filtFD_Im]
```

This function is the same as the `get_CDCfilt_step`, except that FD filters are not symmetrical and after their application, it is not necessary to compensate for time shifts of the signals in different spectral channels (`enable_shift=False`).

5.4.3 get_CDCfilt_JO

Function for joint optimization of all CDC and FD filters.

```
def get_CDCfilt_JO(data_real, filt):
```

This function has the following parameters: array `data_real` consisting of columns with real and imaginary parts of one polarization of one channel and list of arrays with coefficients `filt` for CDC and FD filters calculated using functions `get_CDCfilt_step` and `get_FDfilt`.

```
    data = data_real[0,:] + 1j * data_real[1,:]
    size = len(data)

    delay = num_step * filt_CDC_delay
    if last_step_length > 0:
        delay += filt_CDC_last_delay
    if FD_step_length > 0:
        delay += filt_FD_delay
```

In this block, complex array `data` is formed using the columns of array `data_real`. Variable `size` determines the size of this array. Variable `delay` determines the number of symbols by which the data size on each side will decrease after num_step of linear CDC filters, last CDC and FD filters, if `last_step_length` $>$ 0 and `FD_step_length` $>$ 0.

```
    filtCDC_Re = nd.empty((channels, num_step, filt_CDC_width), dtype=real_t)
    filtCDC_Im = nd.empty((channels, num_step, filt_CDC_width), dtype=real_t)
    if last_step_length > 0:
        filtCDC_last_Re = nd.empty((channels, 1, filt_CDC_last_width), dtype=real_t)
        filtCDC_last_Im = nd.empty((channels, 1, filt_CDC_last_width), dtype=real_t)
    if FD_step_length > 0:
        filtFD_Re = nd.empty((channels, 1, filt_FD_width), dtype=real_t)
        filtFD_Im = nd.empty((channels, 1, filt_FD_width), dtype=real_t)
```

In these lines, arrays for the real and imaginary parts of the coefficients of all CDC and FD filters for all processed channels are initialized.

```
    for i in range(channels):

        netJO = ComplexConvSeq(i)
        for j in range(num_step):
            netJO.add(ComplexConvPredefined(weightsRe=filt[0][i,:,:],
                weightsIm=filt[1][i,:,:], kernel_size=filt_CDC_width,
```

```
                shift=step_shifts[i:i+1,:], dimension=2, pol=1,
                enable_shift=manual_shift, sym=sym_filt))
        if last_step_length > 0:
            netJO.add(ComplexConvPredefined(weightsRe=filt[2][i,:,:],
                weightsIm=filt[3][i,:,:], kernel_size=filt_CDC_last_width,
                shift=last_shifts[i:i+1,:], dimension=2, pol=1,
                enable_shift=manual_shift, sym=sym_filt))
        if FD_step_length > 0:
            netJO.add(ComplexConvPredefined(weightsRe=filt[len(filt)-2][i,:,:],
                weightsIm=filt[len(filt)-1][i,:,:], kernel_size=filt_FD_width,
                dimension=2, pol=1, enable_shift=False, sym=False))
```

In this function, filter coefficients for each spectral channel are also found independently. In this block, the structure of nested neural networks is created using function `ComplexConvSeq()`. The first neural network in this structure will consist of one CDC filter corresponding to the first step. Each next neural network will consist of the previous one with the addition of one convolution layer. Thus, the last network will contain num_step of linear CDC filters, last CDC filter and FD filter. This is necessary so that, during joint optimization, each nested neural network continues to compensate for CD for its corresponding distance.

```
        netJO.initialize(mx.init.Normal(sigma=0.05), ctx=ctx)
        trainer = gluon.Trainer(netJO.collect_params(), 'Adam',
            {'learning_rate': lrate_CDC})
        mse = gluon.loss.L2Loss()

        features = nd.empty((1, 2, size))
        features[:,0,:] = nd.array(np.real(data))
        features[:,1,:] = nd.array(np.imag(data))
        features = features.as_in_context(ctx)
```

This block is similar to the corresponding blocks in the code of the previous two functions.

```
        label_dimension = num_step
        if last_step_length > 0:
            label_dimension += 1
        if FD_step_length > 0:
            label_dimension += 1
        if manual_shift:
            labels = nd.empty((label_dimension,2,size-2*delay-max_full_shift))
        else:
            labels = nd.empty((label_dimension,2,size-2*delay))

        for j in range(num_step):
            data_CDC = cd_operator(data, size, (j + 1) * step_length,
```

```
                Fn[channel_numbers[i]], 'b')
            if manual_shift:
                data_CDC = data_CDC[delay+full_output_shifts[0]:size -
                    (delay+full_output_shifts[1])]
            else:
                data_CDC = data_CDC[delay:size-delay]
            labels[j,0,:] = nd.array(np.real(data_CDC))
            labels[j,1,:] = nd.array(np.imag(data_CDC))
        if last_step_length > 0:
            data_CDC = cd_operator(data, size, (j + 1) * step_length + last_step_length,
                Fn[channel_numbers[i]], 'b')
            if manual_shift:
                data_CDC = data_CDC[delay+full_output_shifts[0]:size -
                    (delay+full_output_shifts[1])]
            else:
                data_CDC = data_CDC[delay:size-delay]
            labels[num_step,0,:] = nd.array(np.real(data_CDC))
            labels[num_step,1,:] = nd.array(np.imag(data_CDC))
        if FD_step_length > 0:
            data_CDC = cd_operator(data, size, fiberL, Fn[channel_numbers[i]], 'b')
            if manual_shift:
                data_CDC = data_CDC[delay+full_output_shifts[0]:size -
                    (delay+full_output_shifts[1])]
            else:
                data_CDC = data_CDC[delay:size-delay]
            labels[label_dimension-1,0,:] = nd.array(np.real(data_CDC))
            labels[label_dimension-1,1,:] = nd.array(np.imag(data_CDC))
        labels = labels.as_in_context(ctx)
```

In this code of block, arrays corresponding to the NN input data (`features`) and the desired NN output (`labels`) are initialized and filled. For structure `netJO` output array should contain arrays obtained after each of filters (output of all nested neural networks). Columns of array `labels` are filled with arrays, which are calculated from `data` by CD compensation for the corresponding distance. These arrays are also transferred to the device `ctx`.

```
        lossCond = True
        epoch = 0
        prev_loss = 10
        while lossCond and epoch < 2000:
            epoch += 1
            tic = time.time()
            train_loss = nd.zeros(1, ctx=ctx)
            with autograd.record():
                output = netJO(features)
                loss = mse(output, labels)
            loss.backward()
```

```
            trainer.step(size)
            train_loss += loss.mean().asscalar()
            loss.wait_to_read()
            if epoch % 100 == 0:
                print('CDC filter sequence -- channel', channel_numbers[i], 'epoch',
                    epoch, '-- loss', train_loss.asscalar(), '-- time', time.time()-tic)
                lossCond = train_loss.asscalar() > 1e-4
                if prev_loss < train_loss.asscalar():
                    break;
                else:
                    prev_loss = train_loss.asscalar()
```

In this block, the neural network is trained in a similar way as in the two previous functions. In this case, the stopping criterion is to achieve the 2000th epoch or reduce losses to 1e-4.

```
        params = netJO.collect_params()
        for j in range(num_step):
            filtCDC_Re[i,j,:] =
                params[list(params)[2*j]].data().reshape(filt_CDC_width,).asnumpy()
            filtCDC_Im[i,j,:] =
                params[list(params)[2*j+1]].data().reshape(filt_CDC_width,).asnumpy()
        if last_step_length > 0:
            filtCDC_last_Re[i,:,:] = params[list(params)[2*num_step]].data()
                .reshape(filt_CDC_last_width,).asnumpy()
            filtCDC_last_Im[i,:,:] = params[list(params)[2*num_step+1]].data()
                .reshape(filt_CDC_last_width,).asnumpy()
        if FD_step_length > 0:
            filtFD_Re[i,:,:] = params[list(params)[len(list(params))-1]].data()
                .reshape(filt_FD_width,).asnumpy()
            filtFD_Im[i,:,:] = params[list(params)[len(list(params))-2]].data()
                .reshape(filt_FD_width,).asnumpy()

    filters = [filtCDC_Re, filtCDC_Im]
    if last_step_length > 0:
            filters = filters + [filtCDC_last_Re, filtCDC_last_Im]
    if FD_step_length > 0:
            filters = filters + [filtFD_Re, filtFD_Im]
    return filters
```

In this block of code, arrays `filtCDC_Re`, `filtCDC_Im`, `filtCDC_last_Re`, `filtCDC_last_Im`, `filtFD_Re` and `filtFD_Im` are filled using the appropriate weights of the trained neural network `netJO`. This function returns list `filters` containing the real and imaginary parts of the coefficients of `num_step` CDC filters, the last CDC filter and the FD filter for all processed channels.

6 Data preparation

Prepare data for further calculations.

```
dir_path = create_metadata()
```

In this line, function `create_metadata` is called to create a file with all run parameters. Variable `dir_path` contains the path to this file.

```
RX = np.empty((channels,data_size), dtype=complex_t)
RY = np.empty((channels,data_size), dtype=complex_t)
```

Here, memory is allocated for complex arrays `RX` and `RY` that correspond to the received symbols for x and y polarizations before CDC.

```
for i in range(channels):
    RX[i,:] = cd_operator(RX_CDC[channel_numbers[i],:], data_size, fiberL,
        Fn[channel_numbers[i]], 'f')
    RY[i,:] = cd_operator(RY_CDC[channel_numbers[i],:], data_size, fiberL,
        Fn[channel_numbers[i]], 'f')
    RX[i,:] /= np.sqrt(np.mean(abs(RX[i,:]) ** 2))
    RY[i,:] /= np.sqrt(np.mean(abs(RY[i,:]) ** 2))
    TX[channel_numbers[i],:] /= np.sqrt(np.mean(abs(TX[channel_numbers[i],:]) ** 2))
    TY[channel_numbers[i],:] /= np.sqrt(np.mean(abs(TY[channel_numbers[i],:]) ** 2))
```

In this code block, first, for each of the considered channels, the arrays `RX` and `RY` are obtained from the arrays `RX_CDC` and `RY_CDC` using the procedure for recovering of signal dispersion broadening. Further, each of the arrays `TX`, `TY`, `RX` and `RY` is normalized so that their average power is equal to 1.

```
del RX_CDC, RY_CDC
```

Here, variables `RX_CDC` and `RY_CDC` are deleted to free memory.

```
X = np.empty((data_dimension, data_size), dtype=real_t)
y = np.empty((data_dimension, data_size), dtype=real_t)

X[::4] = np.real(RX)
X[1::4] = np.imag(RX)
X[2::4] = np.real(RY)
X[3::4] = np.imag(RY)

y[::4] = np.real(TX[channel_numbers,:])
```

```
y[1::4] = np.imag(TX[channel_numbers,:])
y[2::4] = np.real(TY[channel_numbers,:])
y[3::4] = np.imag(TY[channel_numbers,:])
```

In this code block, first memory is allocated for real arrays `X` and `y`, which will later be used for training and testing the neural network. Next, these arrays are filled, and `X` corresponds to the received symbols, while `y` corresponds to the transmitted ones. The order of the columns in these arrays is as follows: real part of x polarization of the first channel under consideration, imaginary part of x polarization of this channel, real part of y polarization of this channel, imaginary part of y polarization of this channel, next the data for the second processed channel in the same order and so on.

```
X_train = X[:,2000:train_data_size+2000]
y_train = y[:,2000:train_data_size+2000]
```

In this block of code, arrays `X_train` and `y_train` are obtained as a slice of `X` and `y` and form a training set. The first 2000 symbols are not used, since there are a lot of errors.

```
del TX, TY, RX, RY
```

In this line, variables `TX`, `TY`, `RX` and `RY` are deleted to free memory.

7 CDC and FD filters coefficients calculation

Predefine jointly optimized coefficients for CDC and FD filters.

```
filt_CDC = get_CDCfilt_step(X_train[:2,:CDC_data_size], filt_CDC_width, step_length,
    step_shifts, step_output_shifts)
```

Here function `get_CDCfilt_step` is called for training complex convolution layers that compensates for CD over a distance `step_length` for each of the channels under consideration. For this, `CDC_data_size` complex symbols of x polarization of the first channel are used. Array `filt_CDC` contains `filt_CDC_width` coefficients corresponding to the real part of the filter, and `filt_CDC_width` coefficients corresponding to the imaginary part for each of the processed channels.

```
if last_step_length > 0:
    filt_CDC_last = get_CDCfilt_step(X_train[:2,:CDC_data_size], filt_CDC_last_width,
        last_step_length, last_shifts, last_output_shifts)
    filt_CDC = filt_CDC + filt_CDC_last
if FD_step_length > 0:
    filt_FD = get_FDfilt(X_train[:2,:CDC_data_size], filt_FD_width, FD_step_length)
    filt_CDC = filt_CDC + filt_FD
```

If `last_step_length` $>$ 0 and `FD_step_length` $>$ 0 then the coefficients for the last CDC and FD filters are found in a similar way. Variable `filt_CDC` is a list of arrays for the found filter coefficients.

```
filt_CDC_JO = get_CDCfilt_JO(X_train[:2,:CDC_data_size], filt_CDC)
```

This line is for a joint optimization of all previously found convolution filters. The resulting array `filt_CDC_JO` is a list containing the real and imaginary parts of the coefficients of `num_step` CDC filters, the last CDC filter and the FD filter for all processed channels.

8 Create and compile model

Create and initialize NN model and dataloaders.

```
tensor_intra = nd.zeros((channels,1,memory_size))
tensor_intra[:,:,nl_mem] = nd.ones((channels,1))

if channels > 1:
    tensor_inter1 = 0.15 * nd.ones(((2*(channels-2)+2),1,memory_size_nc1))
if channels == 4:
    tensor_inter2 = 0.08 * nd.ones((4,1,memory_size_nc2))
    tensor_inter3 = 0.04 * nd.ones((2,1,memory_size_nc3))
```

In this code block, arrays for nonlinear filters initialization are created. For each channel, the nonlinear filter for SPM is initialized with 0 with the exception of 1 in the center of the array (position `nl_mem`). Each of the XPM filters is filled with constants, the values of which were found during numerical calculations. When processing 2 channels, the number of XPM filters for the closest adjacent spectral channels will be 2, for example, a filter that takes into account the nonlinear effect of channel 1 on channel 2, and a filter that takes into account the nonlinear effect of channel 2 on channel 1. When processing 4 channels, there will be 6 such filters. In this case, there will also be 4 XPM filters for spectral channels spaced at one channel spacing and 2 XPM filters spaced at two channel spacing.

```
net = gluon.nn.Sequential()
with net.name_scope():
    for i in range(num_step):
        net.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[0][:,i:i+1,:],
            weightsIm=filt_CDC_JO[1][:,i:i+1,:], kernel_size=filt_CDC_width,
            shift=step_shifts, dimension=data_dimension, enable_shift=manual_shift,
            sym=sym_filt))
        if channels == 1:
            net.add(KerrActivationEnhanced_1ch(dimension=data_dimension,
                tensor_intra=tensor_intra, nl_shifts=nl_shifts, nl_coef=nonlin_coef))
        if channels == 2:
            net.add(KerrActivationEnhanced_2ch(dimension=data_dimension,
                tensor_intra=tensor_intra, tensor_inter=tensor_inter1,
                nl_shifts=nl_shifts, nl_coef=nonlin_coef))
        if channels == 4:
            net.add(KerrActivationEnhanced_4ch(dimension=data_dimension,
                tensor_intra=tensor_intra, tensor_inter1=tensor_inter1,
                tensor_inter2=tensor_inter2, tensor_inter3=tensor_inter3,
                nl_shifts=nl_shifts, nl_coef=nonlin_coef))
    if last_step_length > 0:
        net.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[2], weightsIm=filt_CDC_JO[3],
            kernel_size=filt_CDC_last_width, shift=last_shifts, dimension=data_dimension,
            enable_shift=manual_shift, sym=sym_filt))
    if FD_step_length > 0:
        net.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[len(filt_CDC_JO)-2],
        weightsIm=filt_CDC_JO[len(filt_CDC_JO)-1], kernel_size=filt_FD_width,
        dimension=data_dimension, enable_shift=False))
```

In this block, first the neural network sequential model `net` is initialized. Then `num_step` complex convolution layers and nonlinear activation function layers corresponding to the number of processed channels are added to it. All these layers are initialized with previously defined arrays `filt_CDC_JO`, `tensor_intra` and `tensor_inters`. If necessary, convolution complex layers corresponding to the last CDC and FD filters are also added.

```
net.initialize(init=mx.init.Normal(sigma=0.05), ctx=ctx)
trainer = gluon.Trainer(net.collect_params(), 'Adam', {'learning_rate': lrate})
mse = gluon.loss.L2Loss()
```

Here, all the weights of the neural network are initialized. All parameters for which initial arrays have not been previously defined are initialized with random numbers from normal distribution with `sigma=0.05`. NN weights are located on the device defined in `ctx`. As an optimization algorithm `trainer`, Adam optimizer with standard parameters and a learning rate of `lrate` is used. As a loss function `mse`, the mean squared error (L2 loss) is used.

```
[train_dataloader, batch_train, number_of_bathes_train] = dataloader(X_train, y_train,
    batch_size, True)
del X_train, y_train

[all_data, batch_all, number_of_bathes_all] = dataloader(X, y, batch_size, False)
del X, y
```

In this block of code, objects of the dataloader type for the training set `train_dataloader` and the entire data set `all_data` are created. Variables that contain the size of one batch and the number of batch for each of the set are also created. Variables `X_train`, `y_train`, `X` and `y` are deleted to free memory.

9 Fit the model

Train the neural network.

```
filename = os.path.join(dir_path, model_filename)
model_saved = False
min_loss = 1
```

Variable `filename` contains the path to file with the name `model_filename` located in the folder `dir_path`. The model parameters corresponding to the least reached losses will be saved to this file. Variable `model_saved` indicates whether the model parameters was saved to the file and initially it is equal to `False`. Variable `min_loss` determines the minimum losses reached during training, initially it is equal to 1.

```
decay_steps = 100
steps_wo_min = 0
```

Variable `decay_steps` sets the number of epochs after which there will be a halving of the learning rate if new minimum losses are not reached. This allows us to speed up training in the first epochs and maintain convergence during the training. Variable `steps_wo_min` determines how many epochs `min_loss` was not updated.

```
for epoch in range(epochs):
    train_loss = nd.zeros(1, ctx=ctx)
    tic = time.time()
    for data, label in train_dataloader:
        data = data.as_in_context(ctx)
        label = label.as_in_context(ctx)
        with autograd.record():
```

```
8               output = net(data)
9               loss = mse(output, label)
10              loss = nd.mean(loss)
11
12          loss.backward()
13          trainer.step(1)
14          train_loss += loss.mean().asscalar()
15          loss.wait_to_read()
```

In this code block, the neural network is trained. In a loop for all epochs, first, scalar `train_loss` corresponding to the losses for all batches is declared on the device `ctx`. Variable `tic` contains the start time of each epoch. Then, in a loop for all batches from `train_dataloader`, arrays `data` (NN inputs) and `label` (desired NN output) corresponding to one batch are transferred to the device `ctx` (lines 5 and 6). Command `with autograd.record()` captures code that needs gradients to be calculated. Variable `output` contains an array, which is the result of forward propagation of `data` through the neural network `net`. In lines 9 and 10, the mean squared error `loss` between arrays `output` and `label` is calculated and then averaged over all channels. Command `loss.backward()` compute the gradients with reference to the loss function. The code in line 13 corresponds to the completion one step of optimizer `trainer` parameters update. Next, losses corresponding to the current batch are added to the variable `train_loss`. Command `loss.wait_to_read()` waits until all previous write operations on the current array are finished.

```
1       if (epoch + 1) % 10 == 0:
2           print('epoch', epoch + 1, '-- loss',
3               train_loss.asscalar()/number_of_bathes_train, '-- time', time.time()-tic)
```

This block of code is responsible for printing every 10th epoch the epoch number, loss and runtime of this epoch.

```
1       if min_loss > (train_loss.asscalar()/number_of_bathes_train):
2           min_loss = train_loss.asscalar()/number_of_bathes_train
3           steps_wo_min = 0
4           if save_model_best:
5               net.save_parameters(filename)
6               model_saved = True
7               print('epoch', epoch + 1, '-- Model saved', '-- loss',
8                   train_loss.asscalar()/number_of_bathes_train)
9       else:
10          steps_wo_min += 1
```

Here, if a new minimum of losses has been reached, it is written to `min_loss` and variable `steps_wo_min` is reset to 0. Next, if `save_model_best = True` the parameters of the neural network are written to the file `filename`, variable `model_saved` changes its value to `True` and a message that the model was saved is printed. If a new losses minimum has not been reached, 1 is added to `steps_wo_min`.

```
    if steps_wo_min >= decay_steps:
        steps_wo_min = 0
        lrate /= 2
        if lrate < 1e-5:
            break
        trainer.set_learning_rate(lrate)
        print('epoch', epoch + 1, '-- Learning rate', lrate)

output.wait_to_read()
```

If a new minimum of losses was not reached for `decay_steps` epochs, variable `steps_wo_min` is reset to 0 and parameter of the learning rate is reduced by half. Next, if `lrate` becomes less than `1e-5`, the NN training is terminated, since its continuation is impractical. Otherwise, the new learning rate of optimizer `trainer` is set and a corresponding message is printed.

10 BER calculation

Calculate BER for the entire data set.

```
batch_trunc = batch_all - 2 * full_delay
if manual_shift:
    batch_trunc -= max_full_shift
```

The entire data set is divided into batches of size `batch_all`, and this is the size of the array fed to the NN input. Variable `batch_trunc` contains the size of the array obtained at the output of the neural network and corresponding to one batch (see details in 4.2 and 5.2).

```
y_pred = nd.empty([1, data_dimension, number_of_bathes_all * batch_trunc], dtype=real_t)
y = nd.empty([1, data_dimension, number_of_bathes_all * batch_trunc], dtype=real_t)
```

In this code block memory for real arrays `y_pred` and `y` is allocated. `y_pred` corresponds to NN output for data from all batches from dataloader `all_data`. `y` corresponds to desired NN outputs (`label`) from all batches from `all_data`.

```
if save_model_best and model_saved:
    netTest = gluon.nn.Sequential()
    with netTest.name_scope():
        for i in range(num_step):
            netTest.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[0][:,i:i+1,:],
                weightsIm=filt_CDC_JO[1][:,i:i+1,:], kernel_size=filt_CDC_width,
                shift=step_shifts, dimension=data_dimension, enable_shift=manual_shift,
                sym=sym_filt))
            if channels == 1:
                netTest.add(KerrActivationEnhanced_1ch(dimension=data_dimension,
                    tensor_intra=tensor_intra, nl_shifts=nl_shifts,
                    nl_coef=nonlin_coef))
            if channels == 2:
                netTest.add(KerrActivationEnhanced_2ch(dimension=data_dimension,
                    tensor_intra=tensor_intra, tensor_inter=tensor_inter1,
                    nl_shifts=nl_shifts, nl_coef=nonlin_coef))
            if channels == 4:
                netTest.add(KerrActivationEnhanced_4ch(dimension=data_dimension,
                    tensor_inter1=tensor_inter1, tensor_inter2=tensor_inter2,
                    tensor_inter3=tensor_inter3, tensor_intra=tensor_intra,
                    nl_shifts=nl_shifts, nl_coef=nonlin_coef))
        if last_step_length > 0:
            netTest.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[2],
                weightsIm=filt_CDC_JO[3], kernel_size=filt_CDC_last_width,
                shift=last_shifts, dimension=data_dimension, enable_shift=manual_shift,
                sym=sym_filt))
        if FD_step_length > 0:
            netTest.add(ComplexConvPredefined(weightsRe=filt_CDC_JO[len(filt_CDC_JO)-2],
                weightsIm=filt_CDC_JO[len(filt_CDC_JO)-1], kernel_size=filt_FD_width,
                dimension=data_dimension, enable_shift=False))

    netTest.load_parameters(filename, ctx=ctx)
```

If `save_model_best = True` and the model has been saved, a new neural network model `netTest` is created in the same way as it was done in the section 8. In the last line of this block, all weights of the model `netTest` are initialized with the values of the model parameters stored in the file `filename`. Thus, for BER calculation, the model corresponding to the minimum losses reached will be used instead of the model obtained after the last performed epoch.

```
for batch_idx, data in enumerate(all_data):
    data[0] = data[0].as_in_context(ctx)
    if save_model_best and model_saved:
        output = netTest(data[0])
    else:
        output = net(data[0])
    y_pred[:,:,batch_idx*batch_trunc:(batch_idx+1)*batch_trunc] =
```

```
8           output.as_in_context(mx.cpu(0))
9       y[:,:,batch_idx*batch_trunc:(batch_idx+1)*batch_trunc] = data[1]
10
11  output.wait_to_read()
```

Here, in a loop for all batches from `all_data`, first, array `data[0]` corresponding to NN input is transferred to the device `ctx`. Variable `output` contains an array, which is the result of forward propagation of `data[0]` through the neural network `net` or `netTest` depending on the parameters `save_model_best` and `model_saved`. Next, `output` is transferred to CPU and written to the array `y_pred`. Array `data[1]` corresponding to desired NN output (`label`) from the current batch fills part of the array `y`. Last command in this block waits until all previous write operations are finished.

```
1  y_pred = y_pred.asnumpy()
2  y = y.asnumpy()
```

These lines convert arrays `y_pred` and `y` from MXNet NDArray to NumPy format.

```
1  demap_data_size = 2 ** 20
2  mean_BER_NN = 0
3  for i in range(int(data_dimension/2)):
4      a = demapper(y[:,2*i:2*(i+1),10000:10000+demap_data_size],
5          y_pred[:,2*i:2*(i+1),10000:10000+demap_data_size])
6      mean_BER_NN += calculate_ber(y_pred[:,2*i:2*(i+1),:] * a, y[:,2*i:2*(i+1),:])
7
8  mean_BER_NN /= data_dimension/2
9
10 mean_BER_CDC = (np.mean(BER_CDC_X[channel_numbers]) +
11     np.mean(BER_CDC_Y[channel_numbers]))/2
12 mean_BER_DBP = (np.mean(BER_DBP_X[channel_numbers]) +
13     np.mean(BER_DBP_Y[channel_numbers]))/2
```

In this block of code, BER for the resulting neural network is calculated. Variable `demap_data_size` sets the number of symbols that will be used when calling `demapper`. Arrays `y` and `y_pred` are not fully used to reduce computation time. Variable `mean_BER_NN` corresponding to average BER for the neural network is initialized to zero. In a loop for all channels and polarizations, first, using the function `demapper` parameter `a` that minimizes BER between `y` and `y_pred * a` for the current channel and polarization is found. Next, this parameter is used to calculate BER to the considered polarization and channel and the result is added to `mean_BER_NN`. After the loop variable `mean_BER_NN` is averaged over all channels and polarizations. Next, the average BER for CDC and DBP is calculated using the parameters from the Matlab file (lines 10 - 13).

```
print('BER CDC: ', mean_BER_CDC)
print('BER DBP Huawei: ', mean_BER_DBP)
print('Target BER: ', mean_BER_DBP*0.7)
print('BER NN: ', mean_BER_NN)

f = open(dir_path + 'metadata.md', 'a')
f.write('Epochs to decay: {}\n'.format(decay_steps))
f.write('\nEpochs total: {}\n'.format(epoch + 1))
f.write('Minimum loss: {}\n'.format(min_loss))

f.write('\nBER CDC: {}\n'.format(mean_BER_CDC))
f.write('BER DBP Huawei: {}\n'.format(mean_BER_DBP))
f.write('Target BER: {}\n'.format(mean_BER_DBP*0.7))
f.write('BER NN: {}\n'.format(mean_BER_NN))
```

In this block, all calculated parameters are written to the file `'metadata.md'` and printed.

```
if channels == 1:
    complexity = int(num_step * (0.5 * memory_size + 7 + 4 * filt_CDC_width))

if channels == 2:
    complexity = int(num_step * (0.5 * (memory_size + memory_size_nc1) + 7 +
        4 * filt_CDC_width))

if channels == 4:
    complexity = int(num_step * (0.5 * (memory_size + 1.5 * memory_size_nc1 +
        memory_size_nc2 + 0.5 * memory_size_nc3) + 7 + 4 * filt_CDC_width))

if last_step_length > 0:
    complexity += 4 * filt_CDC_last_width
if FD_step_length > 0:
    complexity += 4 * filt_FD_width


f.write('\nComplexity: {}\n'.format(complexity))
print('Complexity: ', complexity)

f.close()
```

Here, the computational complexity of the resulting neural network is estimated using the formulas presented in ??. Next, the complexity is written to the file `'metadata.md'` and printed.